Argumentation Engine and Strategic Revelation

Exact Semantics, Grounded Mechanisms, and
Reproducible Strategyproofness Experiments

Name : Aashcharya Gorakh **Roll number** : B23ES1001
Name : Damarasingu Akshaya Sree **Roll number** : B23EE1085
Course : Autonomous Systems CSL7650

Abstract

This report presents a self-contained argumentation engine for finite abstract argumentation frameworks and a strategic analysis of argument revelation under grounded semantics. The implementation computes conflict-freeness, defence, characteristic functions, admissibility, complete extensions, grounded extensions, preferred extensions, stable extensions, semi-stable extensions, legal labellings, and acceptance statuses. A grounded direct mechanism evaluates the graph induced by agents' revealed arguments, while exhaustive best-response enumeration determines whether truthful revelation is weakly dominant. A separate parity-based reachability procedure evaluates a conservative interpretation of the sufficient graph condition associated with Theorem 5.4 of the identified textbook.

Experiments evaluate 5,000 randomly generated instances involving three agents and six to eight arguments. Observed strategyproofness fractions are 0.811, 0.181, 0.021, 0.007, and 0.000 for attack probabilities 0.1, 0.3, 0.5, 0.7, and 0.9, respectively. No instance satisfies the implemented sufficient condition while failing the exact strategic test; 709 instances are strategyproof despite failing that condition. The implementation includes a local browser interface, textbook regression cases, replayable manipulation witnesses, and independently checked legal labellings.

Keywords: abstract argumentation; grounded semantics; strategic revelation; strategyproofness; exhaustive verification; multiagent systems.

Contribution and scope

The submission combines three components: an exact semantic engine, a strategic mechanism analyser, and an auditable experimental workflow. Every reported instance classification is obtained by finite exhaustive search, rather than by heuristic optimisation.

The matching source is Chapter 5, "Argumentation Among Agents," in *Multiagent Systems*, second edition, edited by Gerhard Weiss [2, 3]. The assignment omits the required probability values; the five-point grid used here is therefore an explicit experimental assumption.

Public implementation `https://github.com/Aashcharya1/argumentation-engine`

1 Formal Foundations

1.1 Abstract argumentation framework

Following the abstract approach introduced by Dung [1], an argumentation framework is a pair

$$\mathcal{AF} = \langle A, R \rangle, \qquad R \subseteq A \times A, \tag{1}$$

where A is a finite set of arguments and $(a, b) \in R$ means that argument a attacks argument b. The internal logical structure of arguments is outside the model.

For $S \subseteq A$, define its attacked set and the attackers of an argument by

$$S^+ = \{b \in A \mid \exists a \in S : (a, b) \in R\}, \tag{2}$$

$$a^- = \{b \in A \mid (b, a) \in R\}. \tag{3}$$

The framework class uses immutable sets to represent arguments and attacks. It validates attack endpoints and rejects sets containing unknown arguments. Duplicate arguments or attacks have no semantic effect because the underlying mathematical objects are sets.

1.2 Conflict-freeness and defence

A set is conflict-free when it contains no internally attacking pair:

$$\mathrm{CF}(S) \iff S \cap S^+ = \varnothing. \tag{4}$$

A set S defends an argument a when it attacks every attacker of a:

$$\mathrm{Defends}(S, a) \iff a^- \subseteq S^+. \tag{5}$$

An unattacked argument is defended by every set, including the empty set. Defence is evaluated independently of whether the defending set is itself conflict-free.

1.3 Characteristic function

The characteristic function returns precisely the defended arguments:

$$F_{\mathcal{AF}}(S) = \left\{a \in A \mid a^- \subseteq S^+\right\}. \tag{6}$$

This operator is monotone under set inclusion:

$$S \subseteq T \Longrightarrow F_{\mathcal{AF}}(S) \subseteq F_{\mathcal{AF}}(T). \tag{7}$$

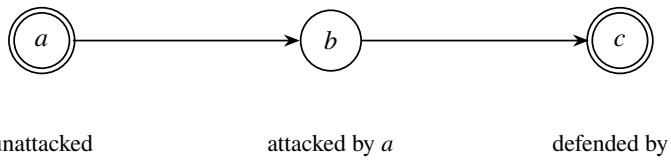


Figure 1. Defence chain. Double circles denote membership in the grounded extension $\{a, c\}$. The set $\{a\}$ defends c by attacking b.

For Figure 1, the characteristic sequence is

$$F^0_{\mathcal{AF}}(\varnothing) = \varnothing, \qquad F^1_{\mathcal{AF}}(\varnothing) = \{a\}, \qquad F^2_{\mathcal{AF}}(\varnothing) = \{a, c\}.$$

The next application makes no change. In particular, $\{a\}$ is admissible but incomplete: it defends an argument that it does not contain.

2 Extension Semantics and Acceptance

2.1 Implemented extension families

Let $\mathrm{rng}(S) = S \cup S^+$ denote the range of a set. The implementation supports the following semantics.

Table 1. Definitions used by the semantic engine.

Concept	Definition or selection rule
Admissible	$\mathrm{CF}(S)$ and $S \subseteq F_{\mathcal{AF}}(S)$.
Complete	$\mathrm{CF}(S)$ and $S = F_{\mathcal{AF}}(S)$.
Grounded	The least complete extension under set inclusion.
Preferred	An inclusion-maximal admissible set.
Stable	$\mathrm{CF}(S)$ and $\mathrm{rng}(S) = A$.
Semi-stable	A complete extension whose range is inclusion-maximal among ranges of complete extensions.

"Maximal" means maximal under inclusion, not greatest cardinality. For example, two incomparable ranges may both be maximal even when their cardinalities differ. The phrase "semi semi-stable" in the assignment is interpreted as semi-stable semantics.

2.2 Legal labellings

For each complete extension E, define

$$L_E(a) = \begin{cases} \mathsf{in}, & a \in E, \\ \mathsf{out}, & a \in E^+, \\ \mathsf{undec}, & a \notin E \cup E^+. \end{cases} \tag{8}$$

The correspondence between complete extensions and legal total labellings allows the engine to generate all legal labellings without testing all $3^{|A|}$ assignments during normal operation.

A legal labelling satisfies the local conditions:

- a is in exactly when every attacker is out;
- a is out exactly when at least one attacker is in;
- otherwise, a is undec.



$$\mathrm{Ext}_{\text{complete}} = \{\varnothing, \{a\}, \{b\}\}$$
$$\mathrm{Ext}_{\text{preferred}} = \mathrm{Ext}_{\text{stable}} = \mathrm{Ext}_{\text{semi-stable}} = \{\{a\}, \{b\}\}$$

Figure 2. Mutual attack produces alternative committed extensions and an all-undecided complete labelling. Its grounded extension is empty.

2.3 Acceptance policy

For a nonempty extension family $\mathcal{E}$, an argument is *skeptically accepted* if it belongs to every extension, *credulously accepted* if it belongs to at least one, and *rejected* if it belongs to none. The API returns the strongest category, so its credulous label means "some but not all."

If a semantics has no extensions, acceptance is reported as *undefined*. This avoids vacuous skeptical acceptance under stable semantics. Rejection concerns extension membership and does not imply that the argument is labelled out in every legal labelling.

3 Exact Semantic Algorithms

3.1 Grounded fixed-point computation

The grounded extension is computed by iterating the characteristic function from the empty set. Monotonicity and finiteness ensure termination.

Algorithm 1 Grounded extension

Input: Finite framework $\mathcal{AF} = \langle A, R \rangle$
Output: Grounded extension $\mathrm{Gr}(\mathcal{AF})$

$S \leftarrow \varnothing$
loop
  $T \leftarrow F_{\mathcal{AF}}(S)$
  if $T = S$ **then**
    return S
  end if
  $S \leftarrow T$
end loop

Every nonterminal iteration adds at least one argument. There are therefore at most $n = |A|$ strict increases, followed by a final equality check.

3.2 Complete, preferred, stable, and semi-stable enumeration

The remaining extension families are computed by enumerating $\mathcal{P}(A)$. Candidate sets are filtered using the definitions in Table 1. Preferred extensions are obtained by removing admissible sets that are strict subsets of other admissible sets. Semi-stable extensions are obtained by comparing ranges of complete extensions.

Algorithm 2 Extension enumeration and maximality filtering

Input: Framework $\mathcal{AF}$ and requested semantics σ

$\mathcal{A} \leftarrow \{S \subseteq A \mid \mathrm{Admissible}(S)\}$
$C \leftarrow \{S \subseteq A \mid \mathrm{Complete}(S)\}$
if σ = complete **then**
  return C
else if σ = preferred **then**
  return $\{S \in \mathcal{A} \mid \nexists T \in \mathcal{A} : S \subsetneq T\}$
else if σ = stable **then**
  return $\{S \subseteq A \mid \mathrm{CF}(S) \wedge \mathrm{rng}(S) = A\}$
else if σ = semi-stable **then**
  return $\{S \in C \mid \nexists T \in C : \mathrm{rng}(S) \subsetneq \mathrm{rng}(T)\}$
end if

Algorithm 2 summarises the selection rules; the code uses separate functions so that requesting one extension family does not require computing every other family.

3.3 Computational scope

With cached attacker sets, characteristic evaluation is polynomial in the graph size. Enumerating subsets introduces a 2^n factor, while pairwise maximality checks can introduce a 4^n factor. Exact enumeration is practical for the assignment's six-to-eight-argument instances, but is not presented as a large-scale solver.

The interactive interface limits inputs to ten arguments to maintain predictable response times. The Python library has no corresponding artificial limit, although its exponential costs remain.

4 Strategic Revelation and Truthfulness

4.1 Mechanism and utility

Let $A_1, \ldots, A_k$ be disjoint ownership sets whose union is A. An agent reports $R_i \subseteq A_i$, including the possibility of revealing nothing. Write

$$U(R) = \bigcup_{i=1}^{k} R_i, \qquad \mathcal{AF}[U] = \langle U, R \cap (U \times U) \rangle.$$

The grounded direct mechanism returns

$$g(R_1, \ldots, R_k) = \mathrm{Gr}\big(\mathcal{AF}[U(R)]\big). \tag{9}$$

The implementation uses the assignment's count utility:

$$u_i(R_i, R_{-i}) = |A_i \cap g(R_i, R_{-i})|\,. \tag{10}$$

Only accepted arguments owned by agent i contribute to its utility.

4.2 Best responses and the exact Theorem 5.3 check

For fixed opponents' reports, the complete best-response set is

$$\mathrm{BR}_i(R_{-i}) = \arg\max_{T_i \subseteq A_i} u_i(T_i, R_{-i}). \tag{11}$$

The best-response routine returns both the maximum utility and *all* maximising reports. Truthfulness is weakly dominant exactly when

$$\forall i,\ \forall R_{-i},\ \forall T_i \subseteq A_i : u_i(A_i, R_{-i}) \geq u_i(T_i, R_{-i}). \tag{12}$$

Algorithm 3 Exact strategyproofness verification

Input: Framework and ownership partition $(A_1, \ldots, A_k)$

for each agent i **do**
  for each $R_{-i} \in \prod_{j \neq i} \mathcal{P}(A_j)$ **do**
    $u_{\text{truth}} \leftarrow u_i(A_i, R_{-i})$
    $(u_{\max}, \mathcal{B}) \leftarrow \text{BestResponses}(i, R_{-i})$
    if $u_{\max} > u_{\text{truth}}$ **then**
      return false, $(i, R_{-i}, u_{\text{truth}}, u_{\max}, \mathcal{B})$
    end if
  end for
end for
return true, no witness

A counterexample permits immediate termination. A positive result requires checking every relevant opponent profile. Ties do not violate strategyproofness.

4.3 Graph certificate and cycle handling

The implemented sufficient condition is

$$\mathrm{C}: \quad \nexists i,\ a, b \in A_i \text{ with a positive odd directed walk from } a \text{ to } b. \tag{13}$$

The test searches a parity-expanded graph with states $(a, 0)$ and $(a, 1)$; each attack flips parity. A visited-state set ensures termination even with cycles.

Here, "walk" explicitly permits repeated vertices and positive odd self-return. This is a conservative interpretation of indirect defeat. It implies the absence of odd simple paths, but its failure may be stronger than failure under a simple-path convention. Consequently, C is reported separately from exact strategyproofness.

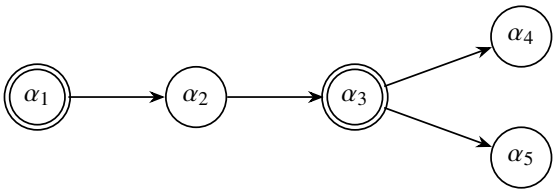
5 Textbook Cases and Corrected Outcomes

The two strategic regression cases follow Examples 5.10 and 5.11 of the identified chapter [3]. In the implementation, `a1` denotes α_1, and similarly for the other arguments.

5.1 Profitable hiding

The first case has ownership

$$A_x = \{\alpha_1, \alpha_4, \alpha_5\}, \qquad A_y = \{\alpha_2\}, \qquad A_z = \{\alpha_3\}.$$



Full revelation: Gr = $\{\alpha_1, \alpha_3\}$

Figure 3. Profitable-hiding case. Agent x can remove the initial defender of α_3 by withholding α_1.

Under truthful reports, utilities are $(u_x, u_y, u_z) = (1, 0, 1)$. If x reveals only $\{\alpha_4, \alpha_5\}$, the outcome becomes $\{\alpha_2, \alpha_4, \alpha_5\}$, giving $u_x = 2$. This is a strict profitable deviation. Utilities are computed using the actual ownership sets, correcting the printed interchange of the other two agents' sets.

5.2 Protective defender

The second case adds α_6 to A_x and the attack $\alpha_6 \to \alpha_3$.

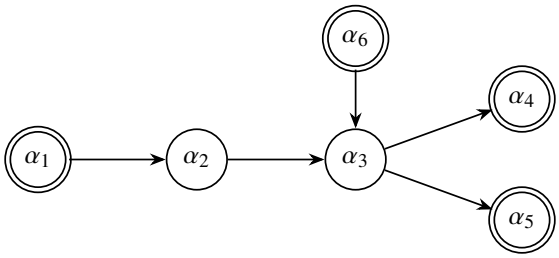


Figure 4. Protective-defender case. The added argument α_6 neutralises α_3. Double circles indicate the truthful grounded outcome.

The truthful outcome is $\{\alpha_1, \alpha_4, \alpha_5, \alpha_6\}$. Exhaustive checking establishes strategyproofness even though internal odd paths remain. The argument cancelling the harmful chain is α_6, rather than α_5 as stated in the erroneous discussion.

This pair of cases separates a graph-level possibility of internal conflict from an actual utility-improving deviation.

6 Experimental Design and Reproducibility

6.1 Sampling protocol

The experiment uses three agents and five attack probabilities:

$$p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}\,.$$

For each probability, 1,000 instances are generated using a single Python pseudorandom generator initialised with seed 20261101.

For every instance:

1. Draw n uniformly from $\{6, 7, 8\}$.
2. For every ordered pair of distinct arguments, include the corresponding attack independently with probability p.
3. Assign each argument independently and uniformly to one of three agents; resample ownership until all agents are nonempty.
4. Evaluate exact strategyproofness using Algorithm 3.
5. Independently evaluate the odd-walk condition in Equation (13).
6. Save the graph, ownership partition, classifications, and available witnesses.

Self-attacks are excluded from the main experiment. Reciprocal attacks, longer directed cycles, and disconnected components are permitted. The library supports self-attacks, and the experiment runner provides an option to include them.

Conditioning on nonempty agents produces a uniform distribution over nonempty ownership assignments for a fixed n; it does not produce uniformly distributed ownership-set sizes.

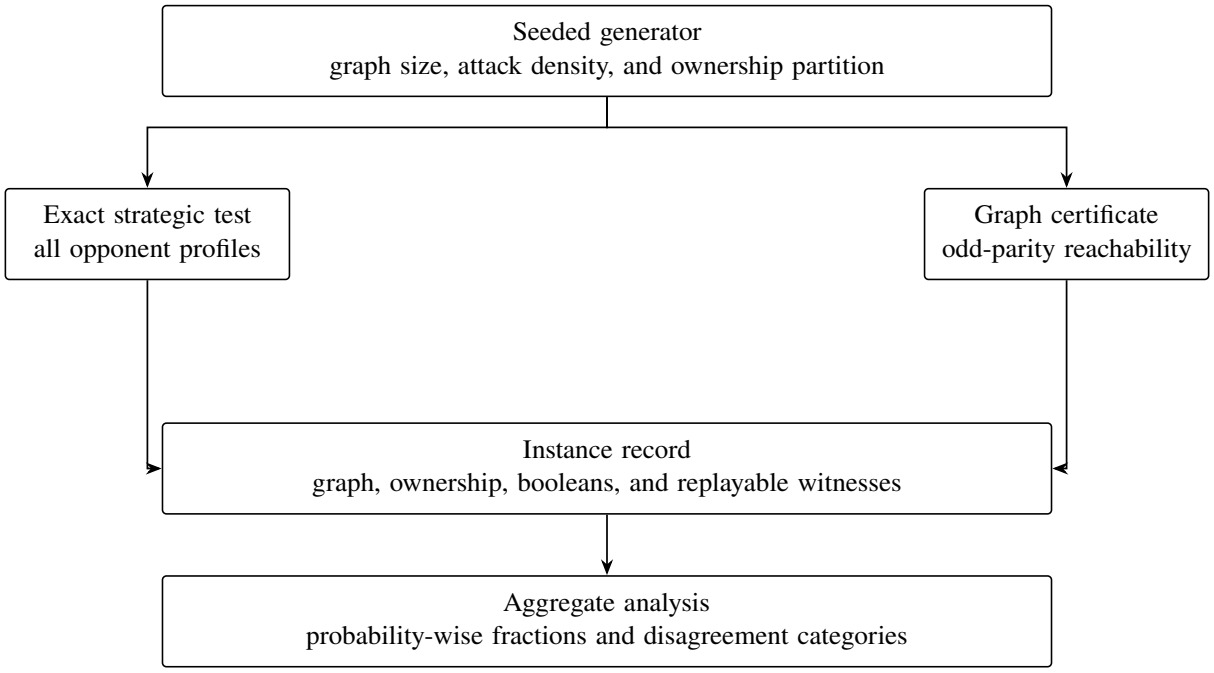


Figure 5. Experimental workflow. Exact strategyproofness is calculated independently of the sufficient graph certificate.

6.2 Audit trail

The saved dataset contains all 5,000 instances, rather than only aggregate counts. A non-strategyproof record includes an agent, opponents' reports, truthful utility, best attainable utility, and optimal alternative reports.

All 3,980 stored profitable-deviation witnesses were replayed successfully. The saved run records a total experiment duration of approximately 8.04 seconds; this is a single-run measurement, not a controlled performance benchmark.

7 Experimental Results

Table 2. Exact classifications over 1,000 instances per attack probability. SP denotes strategyproofness and C the implemented odd-walk condition.

p	$\widehat{\Pr}(\mathrm{SP})$	$\mathrm{SP} \wedge \mathrm{C}$	$\neg\,\mathrm{SP} \wedge \mathrm{C}$	$\mathrm{SP} \wedge \neg\,\mathrm{C}$	$\neg\,\mathrm{SP} \wedge \neg\,\mathrm{C}$
0.1	0.811	297	0	514	189
0.3	0.181	14	0	167	819
0.5	0.021	0	0	21	979
0.7	0.007	0	0	7	993
0.9	0.000	0	0	0	1000
Total	—	311	0	709	3980

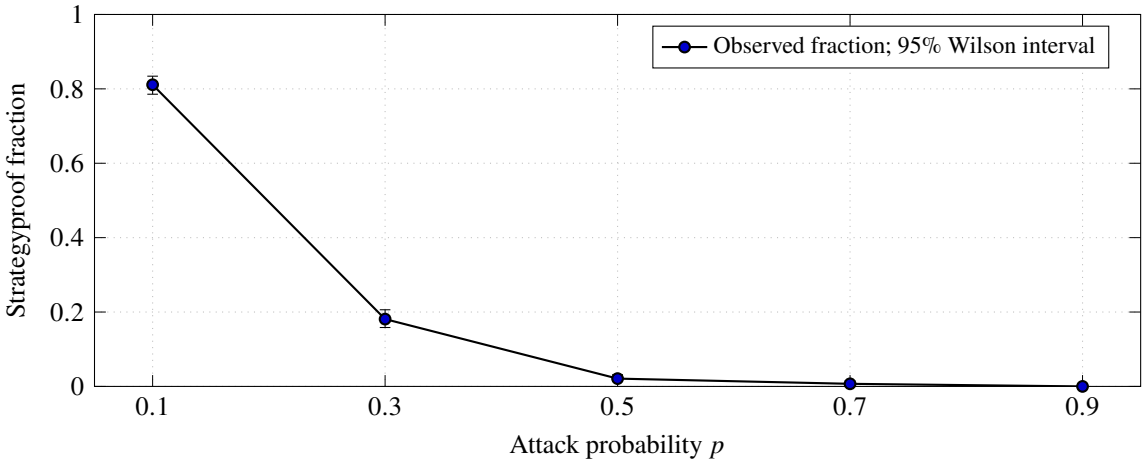


Figure 6. Strategyproofness decreases across the sampled density grid. The connecting line is a visual guide, not an estimated continuous model.

7.1 Condition-passing failures

No instance satisfies C while failing the exact strategic test. There are therefore no observed counterexamples to the implemented sufficient certificate. This finite experiment is consistent with sufficiency; it does not constitute a proof.

7.2 Strategyproof instances outside the certificate

A total of 709 instances satisfy SP $\wedge \neg$ C. Internal odd reachability is therefore not a necessary condition for profitable hiding, nor is its absence a necessary condition for strategyproofness. Potential conflicts may be neutralised by defence, or hiding may exchange accepted arguments without increasing their number.

7.3 Sampling uncertainty

For $N = 1000$ and observed proportion $\hat{q}$, the Wilson interval is

$$\frac{\hat{q} + \dfrac{z^2}{2N} \pm z\sqrt{\dfrac{\hat{q}(1-\hat{q})}{N} + \dfrac{z^2}{4N^2}}}{1 + \dfrac{z^2}{N}}, \qquad z = 1.95996.$$

At $p = 0.9$, zero observed strategyproof instances still gives an upper 95% Wilson endpoint of approximately 0.0038. A zero sample fraction must not be interpreted as a zero population probability.

8 Case Analysis and Interactive System

8.1 An observed strategyproof instance that fails the condition

Saved instance `p0.1-0000` has ownership

$$A_0 = \{b, c\}, \qquad A_1 = \{e, f\}, \qquad A_2 = \{a, d, g, h\}.$$

Its attacks are

$$R = \{(a, b), (a, d), (a, e), (c, d), (e, d), (e, f)\}.$$

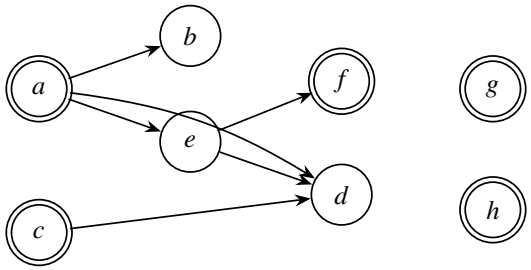


Figure 7. Actual random instance `p0.1-0000`. The truthful grounded extension is $\{a, c, f, g, h\}$; g and h are isolated.

The direct attack $e \to f$ already violates C because both arguments belong to agent 1. If a is revealed, e is defeated and f can be accepted. If a is hidden, revealing e accepts e instead of f. Hiding e can exchange one accepted owned argument for the other, but does not create a strict gain.

Agent 0 cannot influence whether its arguments are attacked: c is unattacked and b depends on the external report of a. Agent 2 can retain its unattacked arguments a, g, h; hiding a can recover at most d while losing a. These structural observations agree with the exhaustive result: the maximum profitable-deviation gain across all profiles is zero.

8.2 Software architecture and manual inspection

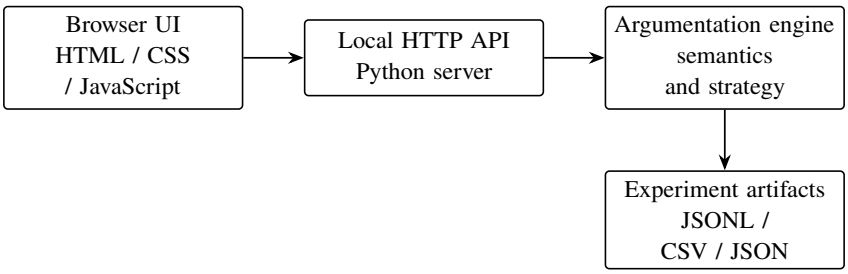


Figure 8. Local system architecture. The UI uses the same Python engine as the executable demonstration and experiment runner.

The interface supports editable attacks, ownership sets, reports, a test set S, all five extension semantics, acceptance tables, legal labellings, and strategic witnesses. Its example menu includes *Profitable hiding* and *Protective defender*. Graph colours in the interactive application communicate acceptance; the figures in this report use monochrome double-circle notation. JSON export and CSV download support manual auditing.

9 Validation, Limitations, and Conclusions

9.1 Verification evidence

The eight automated test cases cover textbook outcomes, grounded defence chains, mutual attacks, odd cycles, self-attacks, invalid inputs, empty frameworks, parity traversal, and a server-analysis response. One test additionally examines *every* directed graph on three arguments, including loops: $2^{3^2} = 512$ graphs.

For each graph, an independent oracle enumerates all $3^3 = 27$ total labellings and checks the local legality conditions directly. These are compared with the labellings derived from complete extensions. Further checks establish grounded containment in every complete extension and membership of stable extensions among preferred extensions.

The saved aggregate table agrees with all 5,000 instance records, and all 3,980 stored profitable deviations replay successfully. These checks provide independent semantic evidence and auditable strategic witnesses; they do not replace formal correctness proofs for arbitrary input sizes.

9.2 Limitations and interpretation

The probability grid and exclusion of self-attacks are documented experimental choices. Results concern disjoint argument ownership, count utility, and hiding-only reports. Overlapping ownership, fabricated arguments, alternative preferences, and other dialogue protocols are outside the implemented model.

The parity test uses positive odd walks, including repeated vertices. A classroom definition restricted to simple paths would require a different certificate test in cyclic graphs. Exact strategyproofness classification remains independent of that choice.

Observed density effects apply to the stated sampling distribution. They are not a monotonicity theorem. Exact enumeration is well suited to the assigned graph sizes but scales exponentially.

9.3 Conclusion and reproducibility

The project supplies an end-to-end implementation connecting formal argumentation semantics with strategic incentives. The experiments separate sufficient graph certificates from exact dominant-strategy truthfulness: no condition-passing failures are observed, while 709 strategyproof instances fall outside the implemented certificate.

```
python -m unittest -v
python demo.py
python experiments.py
python server.py
```

The repository includes the report source material, demonstration guide, complete random-instance records, case studies, and verification notes: `https://github.com/Aashcharya1/argumentation-engine`.

References

[1] P. M. Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and *n*-person games. *Artificial Intelligence*, 77(2):321–357, 1995. doi:10.1016/0004-3702(94)00041-X.

[2] G. Weiss, editor. *Multiagent Systems*. Second edition. MIT Press, 2013. Publisher contents.

[3] I. Rahwan. Argumentation Among Agents. In G. Weiss, editor, *Multiagent Systems*, chapter 5, pp. 177–210. MIT Press, 2013. Author bibliography.

[4] I. Rahwan, K. Larson, and F. Tohmé. A Characterisation of Strategy-Proofness for Grounded Argumentation Semantics. In *Proceedings of IJCAI*, pp. 251–255, 2009. `https://ijcai.org/proceedings/2009/51`.